

Aly Soliman and Alexandre Barbedo

Step 1

We choose option 1.

Status Codes (step 2)

200 OK

Description: The server fulfilled the request.

Relevant methods: All of them.

Trigger Logic

- The request's HTTP version is supported
- The requested resource exists on the server, and the path is correct
- The server has permission to access the resource
- The client does not have the latest version of the resource cached, or there is no version header in the request

Example Test Request

None

```
GET /example.html HTTP/1.1  
Host: localhost
```

304 Not Modified

Description: The resource hasn't changed since the client last fetched it, so the server returns this code to signal to the client to use its cached copy.

Relevant Methods: GET & (maybe) HEAD

Trigger Logic

- The request's HTTP version is supported
- The requested resource exists on the server, and the path is correct
- The server has permission to access the resource
- **The client has a cached copy of the latest version of the resource**

- It requires one of two special headers to know:
 - 'If-Modified-Since' which stores the date that the client last accessed the resource, if the resource hasn't changed since, then 304 is returned
 - 'If-None-Match' The server creates an entity tag for each resource, which changes if it's updated. If the provided ETag matches the current resource, 304 is returned.

None

```
GET /example.html HTTP/1.1
Host: localhost
If-Modified-Since: <IMF-fixdate> OR If-None-Match: <Etag>
```

403 Forbidden

Description: The server refuses to fulfil a request due to a lack of permission.

Relevant Methods: GET, POST, PUT, DELETE

Trigger Logic

- The request's HTTP version is supported
- The requested resource exists on the server, and the path is correct
- **The server does not have permission to access the resource**
 - For example, if a file is not readable

None

```
GET /secured-file.txt HTTP/1.1
Host: localhost
```

404 Not Found

Description: The server fails to fulfil the request because the resource does not exist.

Relevant Methods: GET, POST, PUT, DELETE

Trigger Logic

- The request's HTTP version is supported
- **The requested resource does not exist on the server**

None

```
GET /nonexistent-file.txt HTTP/1.1  
Host: localhost
```

505 HTTP Version Not Supported

Description: The name is pretty self explanatory lol.

Relevant Methods: All of them

Trigger Logic

- **The request's HTTP version is not supported**

None

```
GET /example.html HTTP/3.0  
Host: localhost
```

Step 3

(c) testing the web server

200 OK:

Shell

```
curl.exe -v http://127.0.0.1:8080/test.html  
* Trying 127.0.0.1:8080...  
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from  
127.0.0.1 port 62549  
* using HTTP/1.x  
> GET /test.html HTTP/1.1  
> Host: 127.0.0.1:8080  
> User-Agent: curl/8.21.0  
> Accept: */*
```

```
>
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: text/html
< Content-Length: 308
<
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title></title>
  <meta name="author" content="">
  <meta name="description" content="">
  <meta name="viewport" content="width=device-width,
initial-scale=1">
</head>
<body>
  <p>Congratulations! Your Web Server is Working!</p>
</body>
</html>
* Connection #0 to host 127.0.0.1:8080 left intact
```

404 Not Found:

```
Shell
curl.exe -v http://127.0.0.1:8080/idontexist.html
* Trying 127.0.0.1:8080...
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from
127.0.0.1 port 57830
* using HTTP/1.x
> GET /idontexist.html HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.21.0
```

```
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 404 Not Found
< Content-Type: text/html
< Content-Length: 48
<
<html><body><h1>404 Not Found</h1></body></html>
* Connection #0 to host 127.0.0.1:8080 left intact
```

304 Not Modified:

```
Shell
curl.exe -v -H "If-Modified-Since: Wed, 21 Oct 2030 07:28:00 GMT"
http://127.0.0.1:8080/test.html
* Trying 127.0.0.1:8080...
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from
127.0.0.1 port 57831
* using HTTP/1.x
> GET /test.html HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.21.0
> Accept: */*
> If-Modified-Since: Wed, 21 Oct 2030 07:28:00 GMT
>
* Request completely sent off
< HTTP/1.1 304 Not Modified
<
* Connection #0 to host 127.0.0.1:8080 left intact
```

403 Forbidden:

Shell

```
curl.exe -v http://127.0.0.1:8080/secret.html
* Trying 127.0.0.1:8080...
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from
127.0.0.1 port 57832
* using HTTP/1.x
> GET /secret.html HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Content-Type: text/html
< Content-Length: 48
<
<html><body><h1>403 Forbidden</h1></body></html>
* Connection #0 to host 127.0.0.1:8080 left intact
```

505 HTTP Version Not Supported:

None

```
python 505test.py
HTTP/1.1 505 HTTP Version Not Supported
Content-Type: text/html
Content-Length: 65
<html><body><h1>505 HTTP Version Not Supported</h1></body></html>
```

Performance Evaluation (Step 4)

Proxy Server Efficiency

When handling requests in a proxy server, if the requested pages or files are already cached in the server it would not need to download them from the internet, being more efficient than always downloading requests from the internet.

System Requirements

For a proxy server a Dual-socket architecture is needed. The proxy receives the requests on one socket, parses the destination hostname and opens a secondary outgoing socket to get the data from the web server.

This allows for caching in the proxy server by saving a local copy of the retrieved page or file whenever it takes a request. If other requests are made for the same resource the proxy server sends the file directly from its local cache.

Finally, the proxy can act as a basic firewall and prevent the client from making requests to ports it shouldn't have access to.

Testing and Validation

Test 1: Request Forwarding

Sending a new request to the proxy will forward it to the base server and save the response to cache to directly serve duplicate responses in the future.

This test verifies that sending a new request to the proxy forwards it to the base server and caches the response.

Shell

```
curl.exe -v http://127.0.0.1:8888/test.html
* Trying 127.0.0.1:8888...
* Established connection to 127.0.0.1 (127.0.0.1 port 8888) from
127.0.0.1 port 54323
* using HTTP/1.x
> GET /test.html HTTP/1.1
> Host: 127.0.0.1:8888
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
```

```
< HTTP/1.1 200 OK
< Content-Type: text/html
< Content-Length: 308
<
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title></title>
  <meta name="author" content="">
  <meta name="description" content="">
  <meta name="viewport" content="width=device-width,
initial-scale=1">
</head>
<body>
  <p>Congratulations! Your Web Server is Working!</p>
</body>
</html>
* Connection #0 to host 127.0.0.1:8888 left intact
```

Server Logs:

None

Proxy Output

```
Proxy is listening on http://127.0.0.1:8888
Accepted connection from ('127.0.0.1', 54323)
Proxy Target -> Host: 127.0.0.1 | Port: 8080 | Path: /test.html
Saved response to cache: cache\127.0.0.1_8080_test.html
```

Base Server Output

```
Server is listening on http://127.0.0.1:8080
Accepted connection from ('127.0.0.1', 54324)
Received request: GET /test.html HTTP/1.1
Method: GET | Path: /test.html | Version: HTTP/1.1
```


Test 2: Cached Request

Using the same request to fetch the cached response without communicating with the base server.

Shell

```
curl.exe -v http://127.0.0.1:8888/test.html
* Trying 127.0.0.1:8888...
* Established connection to 127.0.0.1 (127.0.0.1 port 8888) from
127.0.0.1 port 52350
< HTTP/1.1 200 OK
< Content-Type: text/html
... (content served from cache) ...
* Connection #0 to host 127.0.0.1:8888 left intact
```

None

```
# Proxy Output (Cached)
Accepted connection from ('127.0.0.1', 52350)
Proxy Target -> Host: 127.0.0.1 | Port: 8080 | Path: /test.html
Serving cache\127.0.0.1_8080_test.html from local cache.
```

```
# Base Server Output
(No activity recorded on base server)
```

Test 3: Bad Gateway

Attempting to connect to the wrong port should return 502 bad gateway.

Shell

```
curl.exe -v -H "Host: 127.0.0.1:9999"
http://127.0.0.1:8888/test.html
< HTTP/1.1 502 Bad Gateway
< Content-Type: text/html
<html><body><h1>502 Bad Gateway</h1></body></html>
```

None

Proxy Output (Error)

Accepted connection from ('127.0.0.1', 60059)

Proxy Target -> Host: 127.0.0.1 | Port: 9999 | Path: /test.html

Error connecting to target server: [WinError 10061]

Multi-threaded Architecture

This web server is multi-threaded. Whenever the main thread accepts a new incoming socket connection, it launches a new worker thread to handle that client's request and response cycle, and continues to listen for new clients.

By spinning off request handling to worker threads, the main thread never blocks for I/O calls, which allows it to accept and handle new clients much faster. The proxy also follows the same multi-threaded logic so that traffic routed through it benefits from the same performance benefits.

Head-of-Line Blocking Prevention (Step 5)

To prevent Head of Line blocking I had to break every piece of communication the server sends into frames. All messages sent over the TCP socket begin with a 5-byte header. This header contains a stream ID to identify the request pair, a frame type to indicate between header, data or end frames, and a payload length to tell the byte count of the data following.

Because multiple threads try to write to the same TCP socket simultaneously, I implemented a threading lock in the send function. This ensures a thread fully writes a frame before another thread can inject its own frame.

I also created the test file [testFramedClient.py](#) to test the server, as standard web browsers cannot parse custom binary headers.